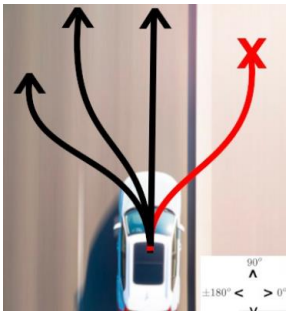
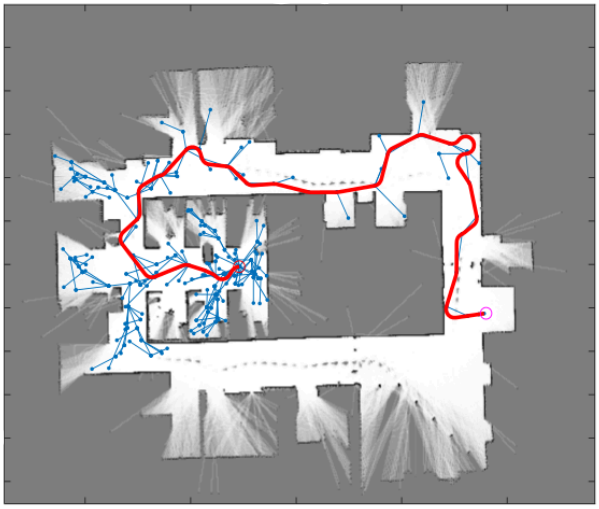
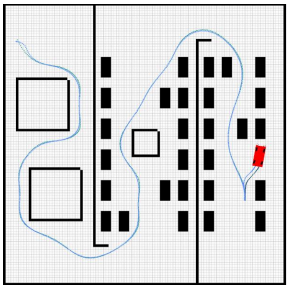
MATLAB / MOTION PLANNING / ACADEMIC PROJECT / 2024

Autonomous Vehicle Path Planning

RRT exploration, route uncertainty, and A* maze search







Dynamic-obstacle planning example from the supplied report. Yellow highlights a final displayed route; green overlays show candidate planning runs.

Project objective

Explore how sampling-based and heuristic-search planners find routes through simplified 2D environments. Study static obstacles, a moving truck with three possible routes, and maze layouts through MATLAB visualization.

My project work

Developed and evaluated MATLAB planning simulations, route-confidence calculations, obstacle scenarios, and result visualizations. Used an attributed MathWorks A* demo as a basis for interactive maze experiments.

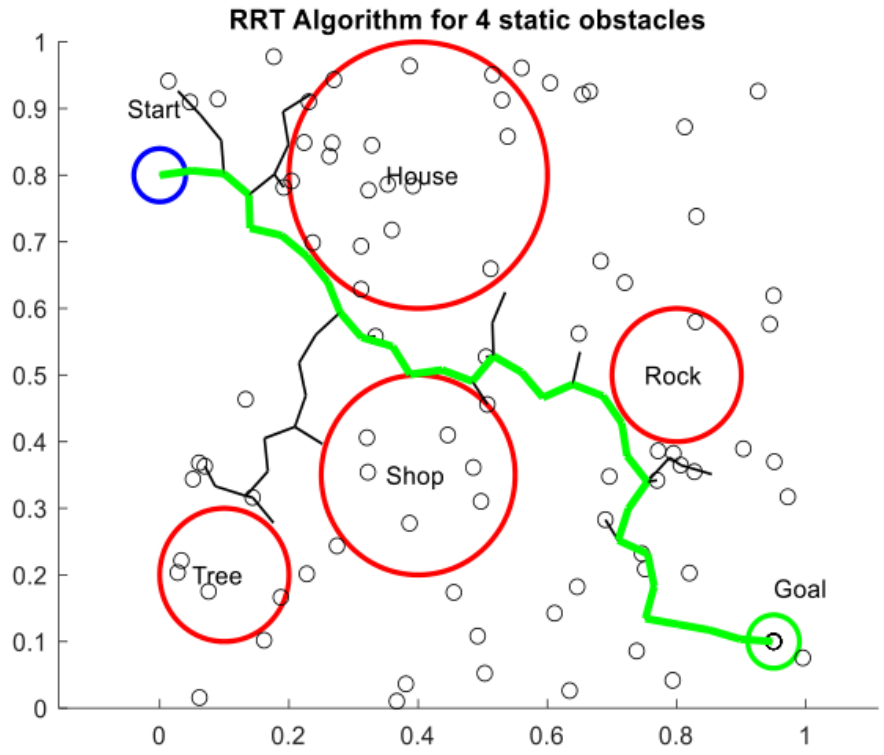
MATLAB | RRT | A* | Bayesian-style inference | 2D simulation

Dakshitha (Dak) Amarasinghe

dakshithaama1994@gmail.com | LinkedIn profile

01 / SAMPLING-BASED SEARCH

RRT around static obstacles



Report page 3: four circular obstacles and the extracted green route. Black branches and sample markers show the exploration process.

Build the search tree

Start at [0, 0.8] and aim for [0.95, 0.1] in a normalized 2D workspace. Sample randomly 90% of the time and select the goal 10% of the time, find the nearest existing node, and extend toward the sample.

Implementation settings

The supplied static example uses epsilon = 0.05, a goal-neighborhood radius of 0.04, and a maximum of 1,000 iterations. Accepted nodes store parent indices so a route can be reconstructed backward.

Collision and termination detail

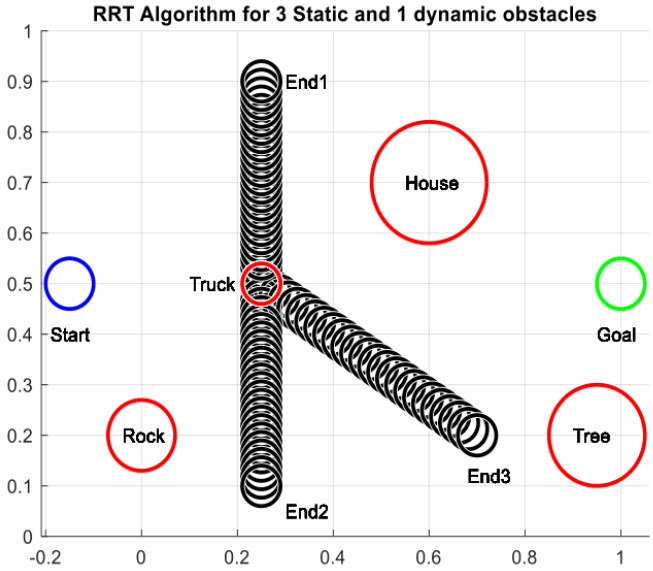
Candidate endpoints are tested against circles; connecting segments and vehicle footprint are not checked. Parent tracing starts at the closest-to-goal node, even if the goal neighborhood was not reached.

Result interpretation: displayed routes are simulation outputs, not a guarantee of collision-free vehicle execution or successful goal arrival on every run.

02 / THREE POSSIBLE TRUCK ROUTES / A

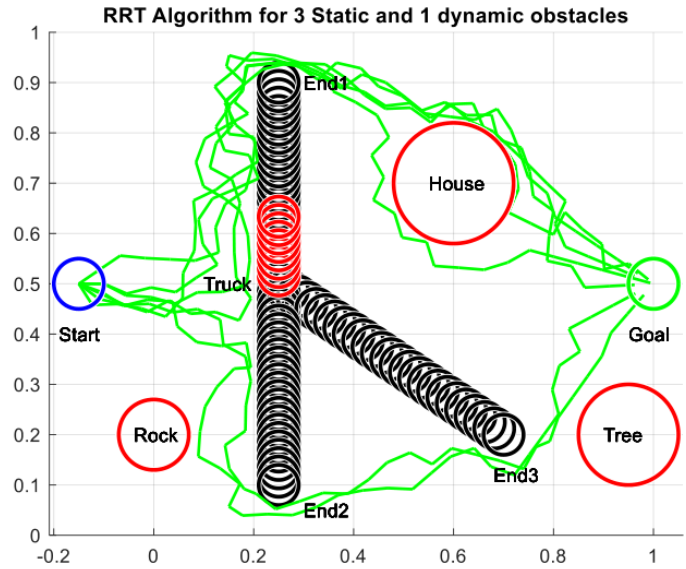
Initial uncertainty to early motion

1. Three hypotheses at the starting position



The truck starts at [0.25, 0.5]. Its candidate destinations are End1 [0.25, 0.9], End2 [0.25, 0.1], and End3 [0.7, 0.2]. Initial trust is 1/3 per route. Black circles mark candidate corridors; the vehicle start and goal lie to either side.

2. Truck begins moving toward End1



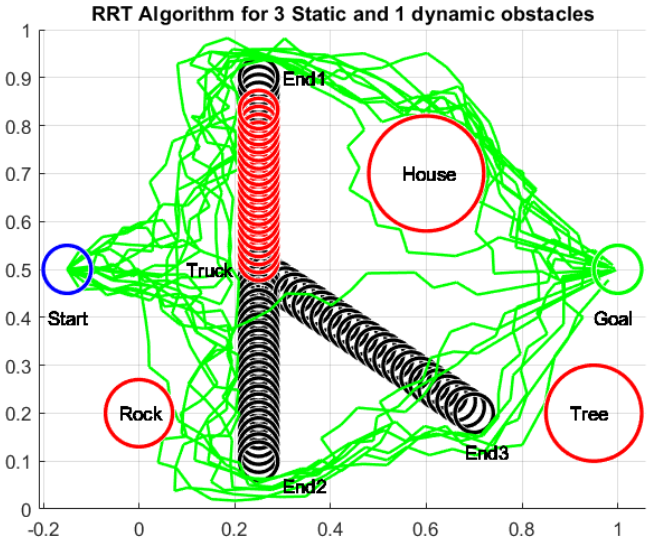
Red circles trace observed truck positions. The report's early-motion view shows green vehicle routes detouring around all three corridors while uncertainty remains. Source order: report page 4, then upper plot on page 5.

Code: truckPath selects one route; each subsequent truck position triggers confidence calculation and a fresh RRTPredict call.

02 / THREE POSSIBLE TRUCK ROUTES / B

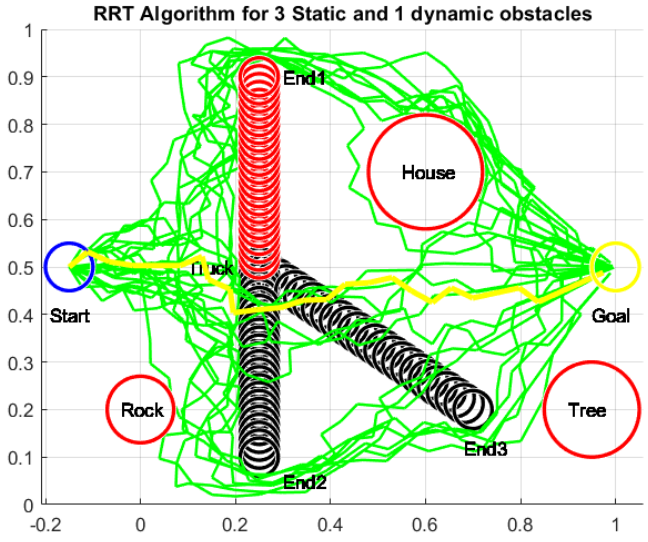
Relaxed restrictions and final route

3. Low-belief corridors become available



The report's next view allows vehicle paths across alternative truck corridors as their beliefs fall below the threshold. The code excludes a corridor only while trust(i) > 0.28, using 20 sampled circles along that route.

4. Final displayed route for End1



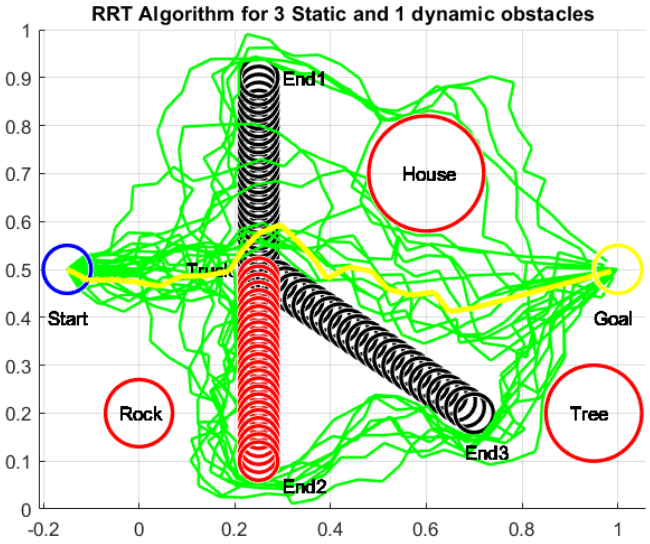
The final RRTPredict call draws its extracted route in yellow. Green overlays retain earlier candidate paths. The yellow goal marker is drawn separately. Source order: lower plot on report page 5, then upper plot on page 6.

Each search restarts from carStart. There is no simulated ego-vehicle motion; nearest-to-goal parent tracing is not proof of arrival.

02 / THREE POSSIBLE TRUCK ROUTES / C

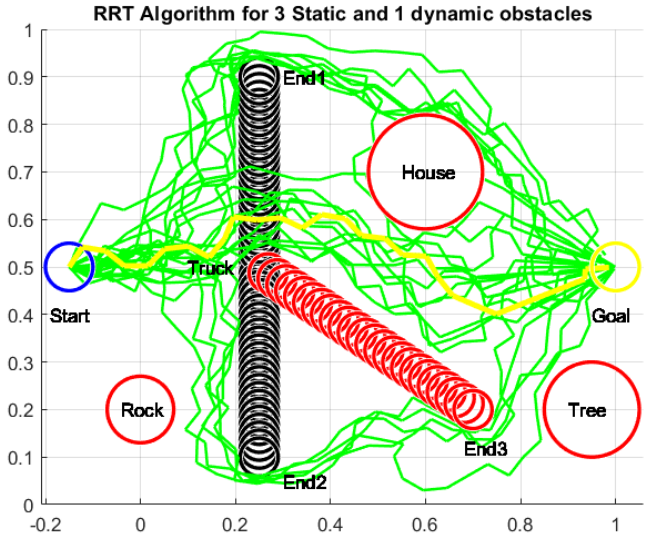
Alternative truck destinations

5. Truck takes the lower route to End2



The report repeats the experiment for a truck heading downward. Its observed positions appear in red along the lower corridor; vehicle candidate paths are replanned against the current route-exclusion mask.

6. Truck takes the diagonal route to End3



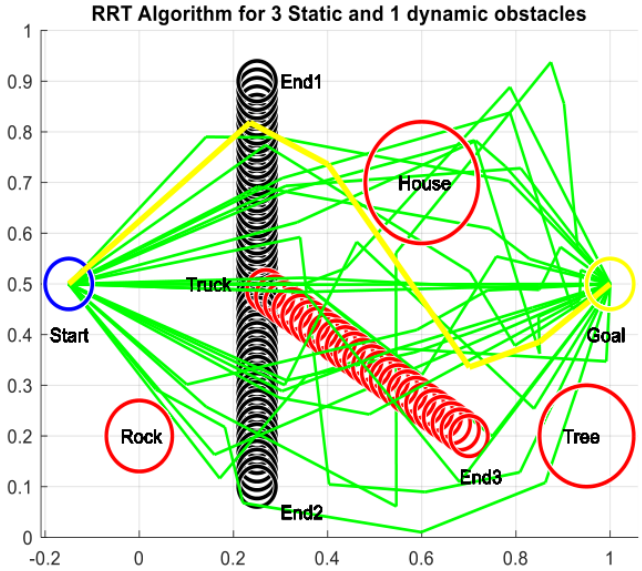
The diagonal-route example uses the same planning logic for End3. In the script, randi([1,3]) selects the route; a commented assignment supports manual selection. Source order: lower plot on report page 6, then page 7.

Trust is recomputed from distance-based scores with a fresh uniform prior, not recursively accumulated from past posteriors.

02 / THREE POSSIBLE TRUCK ROUTES / D

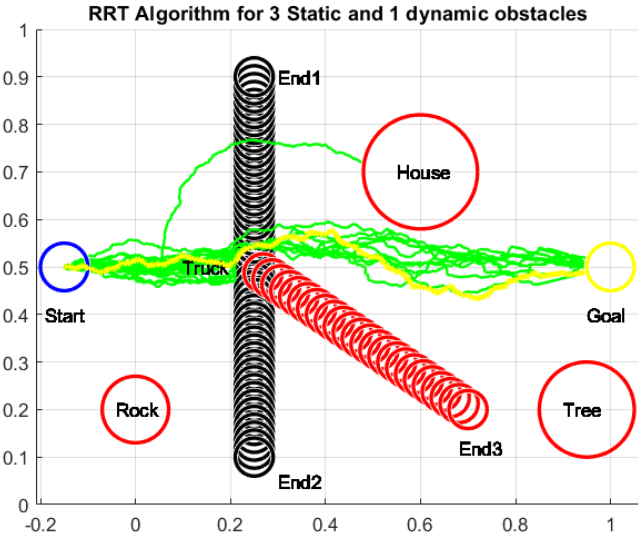
RRT extension-step sensitivity

7. Large epsilon: coarse tree expansion



The report's large-step example shows long, coarse connections. The code checks candidate endpoints against obstacles and excluded corridors; it does not check the complete connecting segment, which is especially important for large steps.

8. Small epsilon: limited search progress



The report's small-step example illustrates restricted progress. With a fixed 1,000-iteration budget, shorter extensions can require more samples to span the workspace. Source: report page 8, upper plot followed by lower plot.

Supplied code sets epsilon=0.05; exact alternative values are not recorded. These are report illustrations, not new benchmark runs.

03 / PROBABILISTIC REASONING

How route beliefs change the search

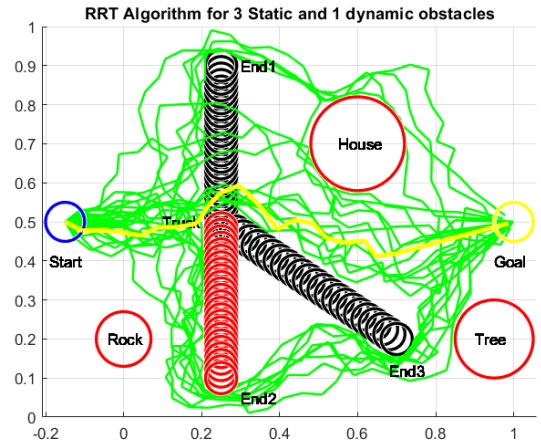
Distance-based evidence

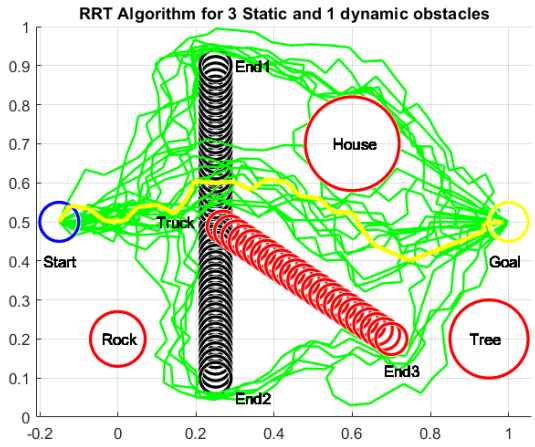
For each candidate truck destination, the code computes a score from the negative distance to the observed truck position, then normalizes exponentiated scores:

L_i = exp(-0.75 * distance(position, goal_i)) / sum_j exp(-0.75 * distance(position, goal_j))

Bayesian-style normalization

The script applies P_i = L_i * prior_i / sum_j(L_j * prior_j). Because every prior is reset to 1/3 at each update, P_i equals L_i in this implementation. Earlier posteriors are not carried forward as new priors.





Alternative truck-route examples from report pages 6-7 show how replanning responds to different selected routes.

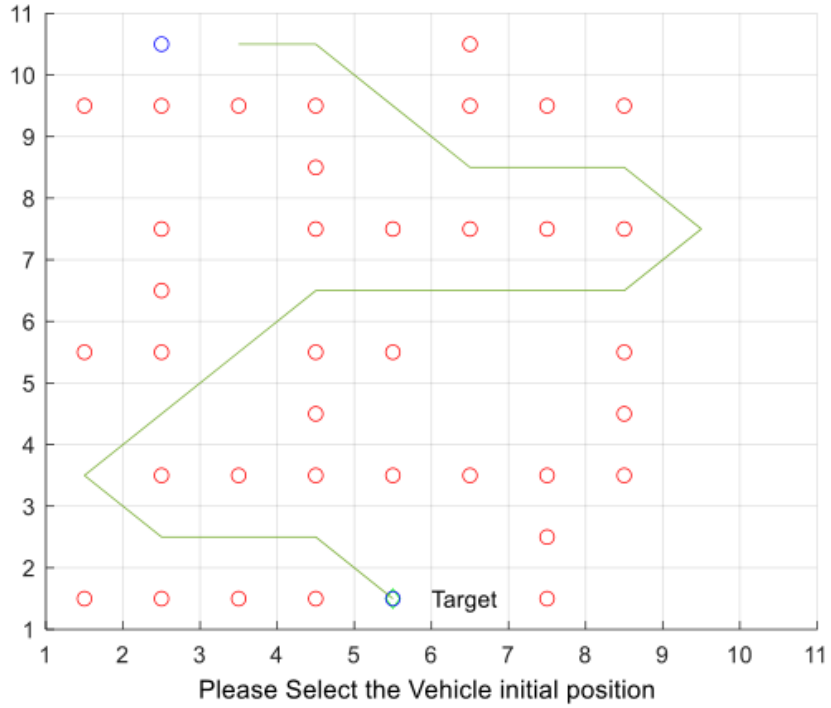
What the threshold means

The value 0.28 controls corridor exclusion in the model. It is a user-selected planning threshold, not a calibrated collision probability or a real-world safety guarantee.

A future extension would carry forward posteriors, use a justified observation model, and assess route-prediction calibration with held-out trajectories.

04 / HEURISTIC GRID PLANNING

Interactive A* maze experiments



Report page 10: a user-defined obstacle layout with the displayed route. Red circles mark blocked grid cells; the route connects the selected start and target.

Grid and search representation

The submitted script creates a 10 x 10 map. Users select the target, obstacles, and initial vehicle location using mouse input. An OPEN list tracks candidate nodes, costs, and parents; a CLOSED list holds obstacles and visited nodes.

Cost model

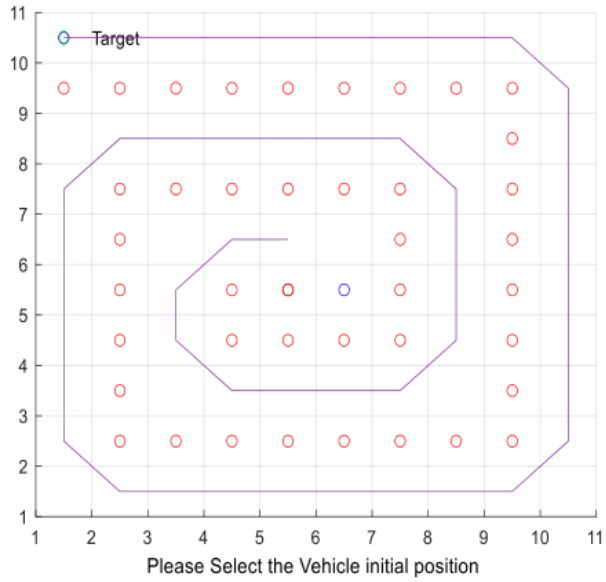
The score f(n) combines accumulated travel cost g(n) with Euclidean heuristic h(n). Eight-connected neighbor expansion allows horizontal, vertical, and diagonal moves; the parent chain reconstructs the route.

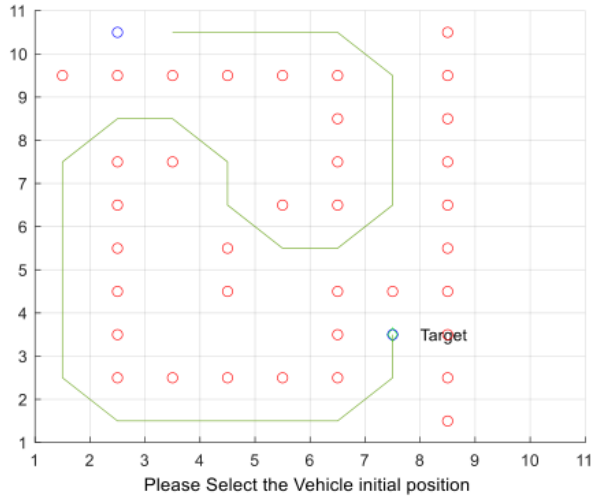
Attribution and scope

The supplied A* file retains a MathWorks demo copyright header and helper-function attribution. This portfolio presents scenario creation, use/adaptation, and evaluation rather than claiming original authorship of that entire implementation.

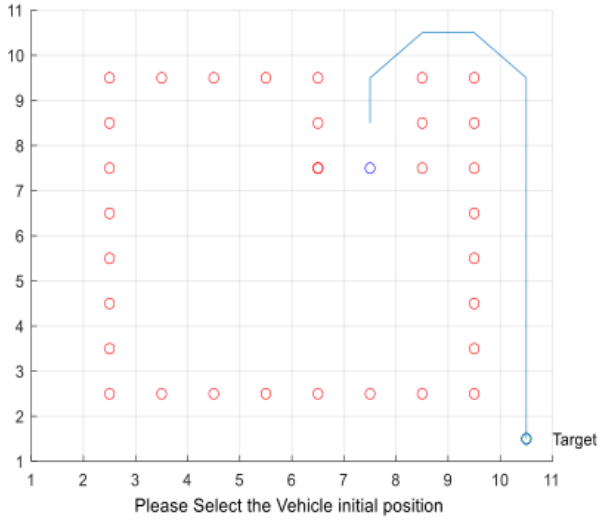
05 / GEOMETRY & SEARCH BEHAVIOR

Different layouts, different routes





Report page 11: spiral-like and curved obstacle patterns with the resulting route overlays.



What these examples show

The experiments illustrate route generation across several deliberately constructed grid layouts. They support a qualitative discussion of geometry and heuristic search.

Important modeling boundary

Diagonal neighbor checks exclude blocked destination cells but do not explicitly prevent corner cutting between blocked adjacent cells. Vehicle dimensions, turn constraints, and dynamics are not modeled.

The report describes favorable A* behavior, but no controlled timing or path-length benchmark establishes universal superiority over RRT.

06 / COMPARISON & NEXT STEPS

What the experiments taught me

RRT: exploration and parameter sensitivity

Random sampling can generate different routes on repeated runs. The report records sensitivity to epsilon and narrow passages in the tested setup. These are observations of this implementation and environment, not proof that RRT always fails in narrow gaps.

A*: structured search and movement rules

Grid search makes obstacle layouts and cost behavior easy to inspect. Path quality depends on the heuristic, connectivity, and valid-transition rules; shortest-path guarantees should not be extended to an unmodeled physical vehicle.

Uncertainty changes usable space

The truck example connects motion hypotheses to planning constraints. Distance-based route confidence relaxes some exclusion corridors as one destination becomes more plausible, illustrating a perception/prediction/planning relationship.

Next implementation steps

Validate every RRT edge and goal connection; inflate obstacles for vehicle footprint; return explicit success/failure; prevent A* corner cutting; add fixed random seeds and repeated trials; and compare success rate, planning time, path length, and minimum clearance.

Evidence and attribution

Prepared from two supplied MATLAB scripts, the 13-page final report, and the project presentation, dated March 2024. Visuals are reproduced from the report; no new planning runs were performed for this portfolio. The report also includes an occupancy-grid RRT example beyond the custom circular-obstacle script.

Delivered scope: academic 2D planning experiments and visualization. This project does not establish deployed autonomous driving, certified safety, or measured real-world navigation accuracy.